

Tetuan City Power Consumption Analysis

Yasmin Guadarrama, Harsh Mishrikotkar, Shivanshi Shukla

2026-05-05

Contents

Section 1: Data Setup, Cleaning and Processing	2
Section 2: Feature Engineering	7
Section 3: Data Standardization	8
Section 4: Predictive Modeling	9
4.1 Baseline Linear Regression Models	9
4.2 Regularized Regression Models (Ridge, Lasso, Elastic Net)	11
4.3 Random Forest Models	12
4.4 Gradient Boosting Models (XGBoost)	12
Section 5: Model Tuning and Cross-Validation	13
Section 6: Model Evaluation	15
Zone 1	15
Zone 2	15
Zone 3	16
Overall Comparison	16
Section 7: Interpretation of Results	17
7.1 Importance of Model Choice	17
7.2 Limitations of the Current Feature Set	17
7.3 Model Complexity Does Not Guarantee Better Performance	17
7.4 Implications for Prediction	17
7.5 Recommendations for Improvement	17
Section 8: Project Workflow and Contributions	18
Appendix	19

Section 1: Data Setup, Cleaning and Processing

```
## Rows: 52416 Columns: 9
## -- Column specification -----
## Delimiter: ","
## chr (1): DateTime
## dbl (8): Temperature, Humidity, Wind Speed, general diffuse flows, diffuse f...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

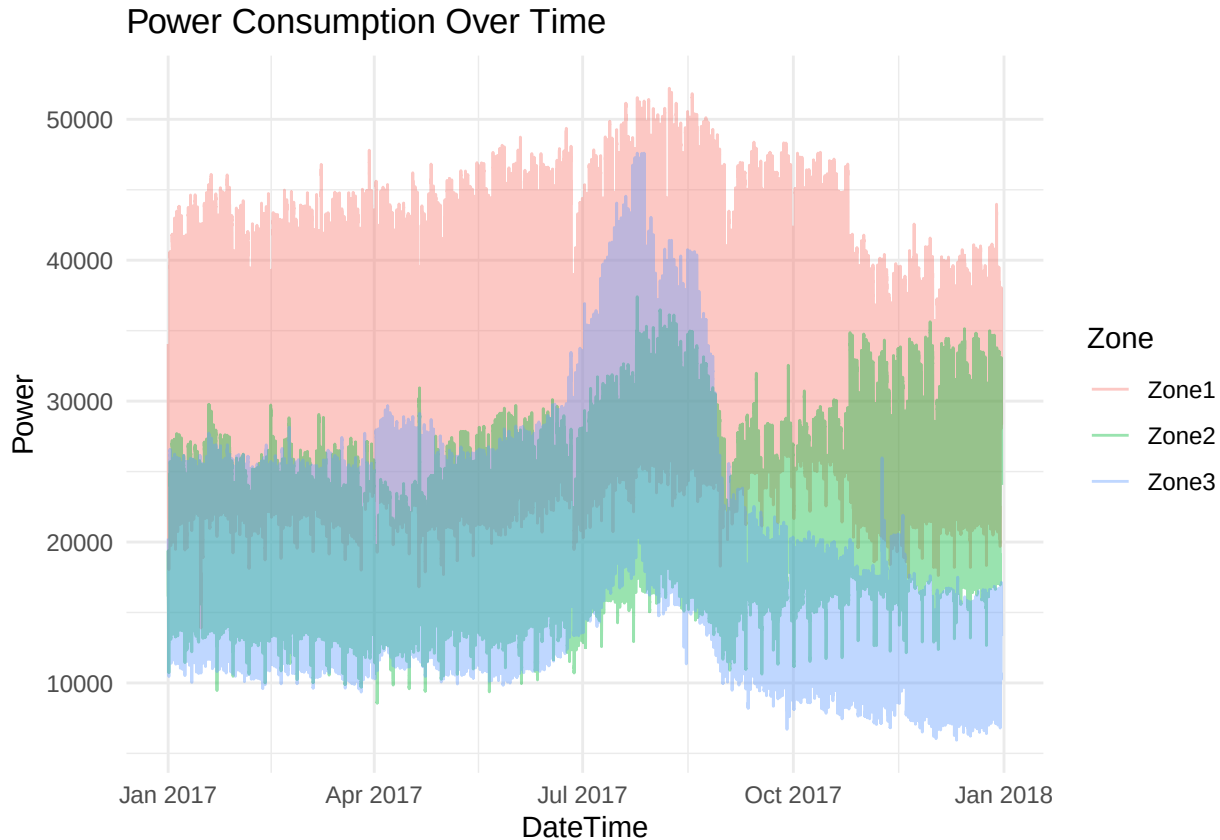
```
## Rows: 52,416
## Columns: 9
## $ DateTime                <chr> "1/1/2017 0:00", "1/1/2017 0:10", "1/1/201~
## $ Temperature            <dbl> 6.559, 6.414, 6.313, 6.121, 5.921, 5.853, ~
## $ Humidity                <dbl> 73.8, 74.5, 74.5, 75.0, 75.7, 76.9, 77.7, ~
## $ 'Wind Speed'           <dbl> 0.083, 0.083, 0.080, 0.083, 0.081, 0.081, ~
## $ 'general diffuse flows' <dbl> 0.051, 0.070, 0.062, 0.091, 0.048, 0.059, ~
## $ 'diffuse flows'        <dbl> 0.119, 0.085, 0.100, 0.096, 0.085, 0.108, ~
## $ 'Zone 1 Power Consumption' <dbl> 34055.70, 29814.68, 29128.10, 28228.86, 27~
## $ 'Zone 2 Power Consumption' <dbl> 16128.88, 19375.08, 19006.69, 18361.09, 17~
## $ 'Zone 3 Power Consumption' <dbl> 20240.96, 20131.08, 19668.43, 18899.28, 18~
```

```
##      DateTime      Temperature      Humidity      Wind Speed
## Length :52416      Min.      : 3.247      Min.      :11.34      Min.      :0.050
## N.unique :52416      1st Qu.:14.410      1st Qu.:58.31      1st Qu.:0.078
## N.blank  :      0      Median :18.780      Median :69.86      Median :0.086
## Min.nchar:   13      Mean      :18.810      Mean      :68.26      Mean      :1.959
## Max.nchar:   16      3rd Qu.:22.890      3rd Qu.:81.40      3rd Qu.:4.915
## Max.      :40.010      Max.      :94.80      Max.      :6.483
## general diffuse flows diffuse flows      Zone 1 Power Consumption
## Min.      : 0.004      Min.      : 0.011      Min.      :13896
## 1st Qu.: 0.062      1st Qu.: 0.122      1st Qu.:26311
## Median : 5.035      Median : 4.456      Median :32266
## Mean      :182.697      Mean      : 75.028      Mean      :32345
## 3rd Qu.: 319.600      3rd Qu.:101.000      3rd Qu.:37309
## Max.      :1163.000      Max.      :936.000      Max.      :52204
## Zone 2 Power Consumption Zone 3 Power Consumption
## Min.      : 8560      Min.      : 5935
## 1st Qu.:16981      1st Qu.:13129
## Median :20823      Median :16415
## Mean      :21043      Mean      :17835
## 3rd Qu.:24714      3rd Qu.:21624
## Max.      :37409      Max.      :47598
```

```
## [1] 0
```

```
## [1] 0
```

```
## [1] "2017-01-01 00:00:00 UTC" "2017-12-30 23:50:00 UTC"
```



This line graph tracks power consumption for Zone 1, Zone 2, and Zone 3 across the entire year of 2017.

All three zones show repeating peaks and dips, indicating predictable usage patterns tied to human activity.

There are noticeable increases during mid-year (summer months), likely due to cooling demand.

Zone 1 consistently has the highest demand.

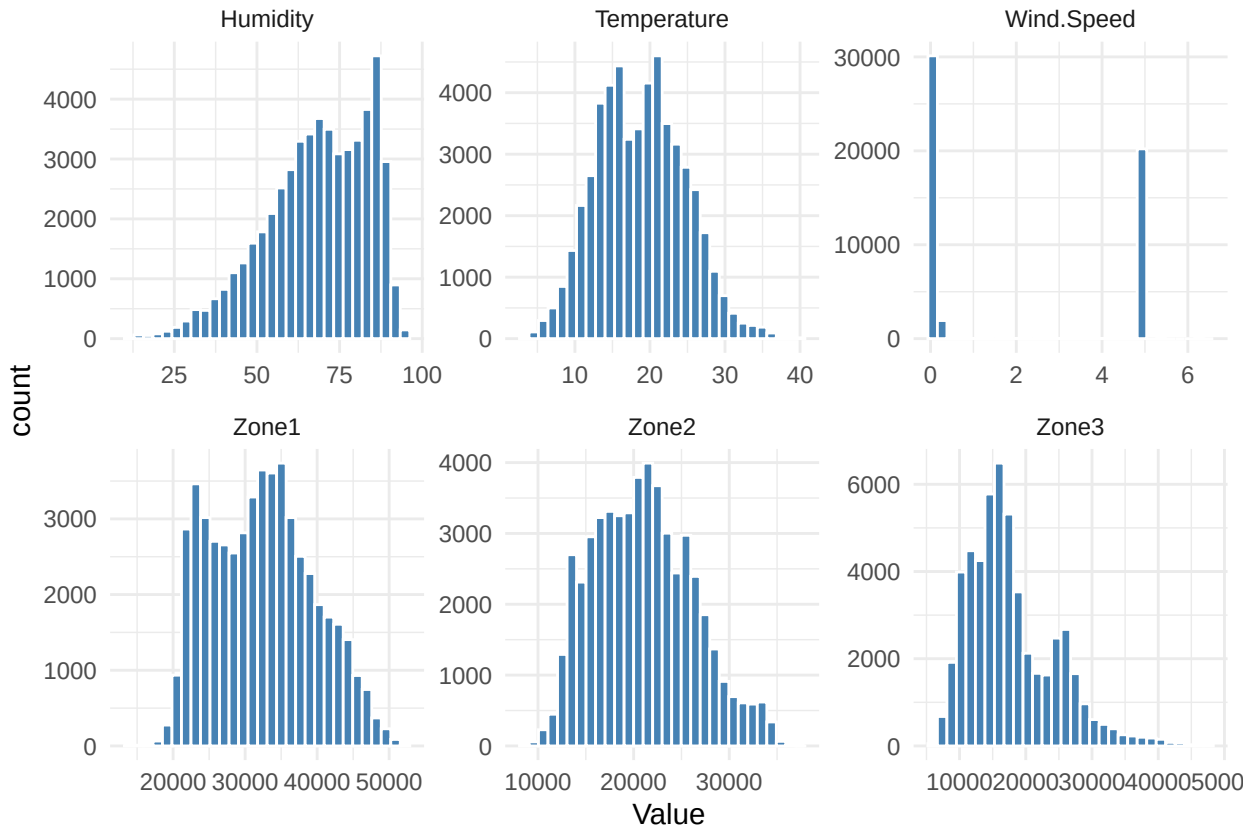
Zone 2 is moderate.

Zone 3 is the lowest.

This suggests different population densities or industrial loads.

The clear temporal structure justifies adding hour, day of week, and month as predictors.

Models capturing nonlinear patterns (Random Forest, Boosting) may perform better than simple linear regression.



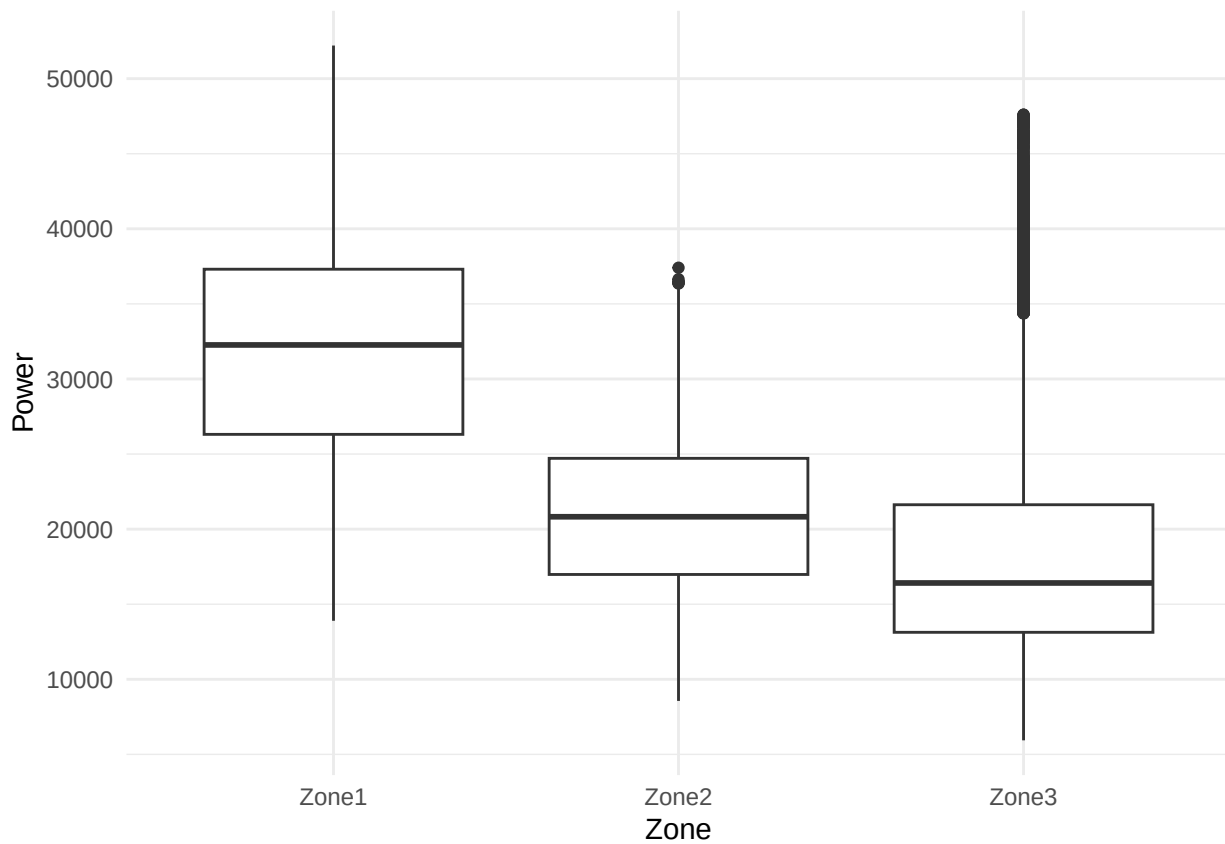
There are six histograms: Humidity, Temperature, Wind Speed, Zone 1, Zone 2, Zone 3.

Both have bell-shaped distributions, meaning they vary smoothly around typical weather conditions. Temperature ranges from $\sim 3^{\circ}\text{C}$ to $\sim 40^{\circ}\text{C}$, showing clear seasonal spread.

This suggests Tetouan experiences both calm periods and windy periods.

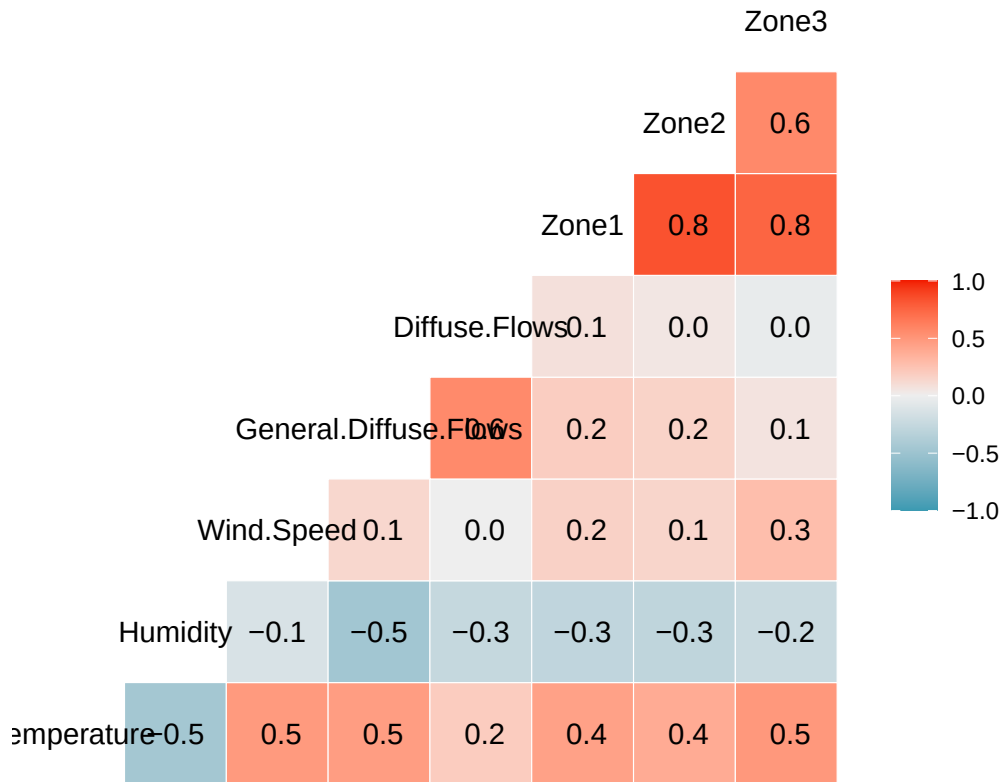
Zone Power Consumption (1, 2, 3) All three zones have right-skewed distributions. This means most consumption values are moderate, with occasional high-demand spikes.

Skewed target variables may benefit from models that handle nonlinear relationships. Weather variables appear well-behaved and suitable for regression.



Zone 1 has the highest median and widest spread, the largest and most variable demand. Zone 2 is lower than Zone 1 but still substantial. Zone 3 has the lowest median and the smallest spread. Outliers appear in Zone 2 and Zone 3, indicating occasional spikes.

The zones differ significantly in their baseline consumption levels. This supports modeling each zone separately rather than combining them. Different zones may require different model complexities or may respond differently to weather/time predictors.



This heatmap shows pairwise correlations among weather variables and zone consumption.

Strong positive correlations: Zone 1 - Zone 2 - Zone 3 (0.7–0.8) All zones rise and fall together - likely due to shared city-wide activity patterns.

Temperature vs. Humidity: Moderately negative correlation (~ -0.5). Hotter days tend to be less humid.

Diffuse flows vs. general diffuse flows: Very strong positive correlation. These two solar radiation measures move together.

Temperature vs. Power Consumption: Mild positive correlation. Warmer weather increases electricity use (likely cooling).

High correlation among zones suggests similar predictors will work for all three. Correlated predictors (like the two diffuse flow variables) may cause multicollinearity in linear regression, LASSO/Ridge can help. Weather variables show meaningful relationships with power consumption, supporting their inclusion.

Section 2: Feature Engineering

To capture temporal patterns more effectively, we extract features such as hour, day of week, and month from the datetime variable. These are then transformed using cyclical encoding (sine and cosine functions). This approach preserves the natural periodic structure of time (e.g., hour 23 is close to hour 0), which improves the ability of machine learning models to detect repeating patterns.

Section 3: Data Standardization

All numerical predictors are standardized to have mean 0 and standard deviation 1. This ensures that variables are on a comparable scale, which is particularly important for models sensitive to feature magnitude, such as regression and distance-based algorithms.

Section 4. Train-Test Split

The dataset is split into training (80%) and testing (20%) sets using a time-based split rather than random sampling. This approach preserves the chronological order of observations and prevents data leakage, ensuring that the model is evaluated on future data rather than past observations.

Section 4: Predictive Modeling

In this section, we develop and compare multiple predictive models for forecasting power consumption in each zone. We begin with baseline linear regression models and progressively introduce more flexible machine learning approaches, including regularized regression, tree-based models, and gradient boosting. Model performance is evaluated using standard regression techniques, and cross-validation is used where appropriate to improve generalization.

4.1 Baseline Linear Regression Models

Before applying more complex machine learning techniques, we first construct baseline multiple linear regression models for each zone. These models serve as a benchmark for evaluating whether more advanced methods provide meaningful improvements. Categorical variables such as day of week and month are converted into factors to ensure correct treatment in the regression framework.

```
##
## Call:
## lm(formula = Zone1 ~ Temperature + Humidity + Wind.Speed + General.Diffuse.Flows +
##     Diffuse.Flows + hour + dow + month, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -16374.3  -2940.8   -254.1    2918.4   19481.1
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    23056.01     115.10  200.319  < 2e-16 ***
## Temperature     1861.95      58.96   31.579  < 2e-16 ***
## Humidity        -12.98      28.37   -0.458  0.647169
## Wind.Speed     -190.52      26.80   -7.109  1.18e-12 ***
## General.Diffuse.Flows -80.74      28.86   -2.797  0.005155 **
## Diffuse.Flows   -534.12      24.59  -21.721  < 2e-16 ***
## hour           717.77        3.30  217.485  < 2e-16 ***
## dow2          1762.39       78.48   22.457  < 2e-16 ***
## dow3          1872.21       78.47   23.859  < 2e-16 ***
## dow4          2060.64       78.54   26.237  < 2e-16 ***
## dow5          1962.01       78.87   24.877  < 2e-16 ***
## dow6          1527.21       79.06   19.317  < 2e-16 ***
## dow7          1621.78       78.86   20.566  < 2e-16 ***
## month2         88.25        93.85    0.940  0.347042
## month3        -368.15       93.52   -3.937  8.27e-05 ***
## month4        -980.25      102.23   -9.588  < 2e-16 ***
## month5       -705.29      116.44   -6.057  1.40e-09 ***
## month6         792.59      128.42    6.172  6.82e-10 ***
## month7         572.65      162.75    3.519  0.000434 ***
## month8        1506.97      158.31    9.519  < 2e-16 ***
```

```

## month9          -714.03      133.87  -5.334 9.67e-08 ***
## month10         31.65       136.30   0.232 0.816364
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4306 on 41910 degrees of freedom
## Multiple R-squared:  0.6399, Adjusted R-squared:  0.6397
## F-statistic: 3546 on 21 and 41910 DF,  p-value: < 2.2e-16

##
## Call:
## lm(formula = Zone2 ~ Temperature + Humidity + Wind.Speed + General.Diffuse.Flows +
##     Diffuse.Flows + hour + dow + month, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -10537  -1910   -120    1895   11438
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    13830.770      76.476  180.850 < 2e-16 ***
## Temperature     1304.210      39.177   33.290 < 2e-16 ***
## Humidity         42.599      18.848    2.260  0.0238 *
## Wind.Speed     -205.411      17.806  -11.536 < 2e-16 ***
## General.Diffuse.Flows  115.420      19.178    6.018 1.78e-09 ***
## Diffuse.Flows  -266.821      16.339  -16.330 < 2e-16 ***
## hour           430.119       2.193  196.141 < 2e-16 ***
## dow2           2148.334      52.146   41.198 < 2e-16 ***
## dow3           2383.543      52.141   45.714 < 2e-16 ***
## dow4           2478.934      52.185   47.503 < 2e-16 ***
## dow5           2677.573      52.405   51.094 < 2e-16 ***
## dow6           2365.262      52.533   45.024 < 2e-16 ***
## dow7           1456.068      52.396   27.790 < 2e-16 ***
## month2         -546.864      62.359   -8.770 < 2e-16 ***
## month3        -1406.092      62.138  -22.628 < 2e-16 ***
## month4        -2603.277      67.930  -38.323 < 2e-16 ***
## month5        -1010.078      77.372  -13.055 < 2e-16 ***
## month6         -839.033      85.331   -9.833 < 2e-16 ***
## month7         1843.784     108.139   17.050 < 2e-16 ***
## month8         2531.729     105.187   24.069 < 2e-16 ***
## month9        -1358.469      88.950  -15.272 < 2e-16 ***
## month10        -486.014      90.563   -5.367 8.07e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2861 on 41910 degrees of freedom
## Multiple R-squared:  0.6689, Adjusted R-squared:  0.6687
## F-statistic: 4032 on 21 and 41910 DF,  p-value: < 2.2e-16

##
## Call:
## lm(formula = Zone3 ~ Temperature + Humidity + Wind.Speed + General.Diffuse.Flows +
##     Diffuse.Flows + hour + dow + month, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max

```

```
## -12900 -2161 38 2135 14835
##
## Coefficients:
## Estimate Std. Error t value Pr(>|t|)
## (Intercept) 14341.677 92.128 155.671 < 2e-16 ***
## Temperature 1904.198 47.195 40.347 < 2e-16 ***
## Humidity -213.232 22.705 -9.391 < 2e-16 ***
## Wind.Speed -45.743 21.451 -2.132 0.03297 *
## General.Diffuse.Flows -1117.293 23.102 -48.363 < 2e-16 ***
## Diffuse.Flows -874.156 19.683 -44.412 < 2e-16 ***
## hour 445.538 2.642 168.655 < 2e-16 ***
## dow2 -235.290 62.819 -3.746 0.00018 ***
## dow3 -34.456 62.812 -0.549 0.58331
## dow4 -119.329 62.866 -1.898 0.05768 .
## dow5 -262.549 63.130 -4.159 3.20e-05 ***
## dow6 -447.134 63.284 -7.065 1.63e-12 ***
## dow7 107.654 63.120 1.706 0.08810 .
## month2 -133.908 75.122 -1.783 0.07467 .
## month3 -830.957 74.855 -11.101 < 2e-16 ***
## month4 59.710 81.832 0.730 0.46560
## month5 -1454.420 93.208 -15.604 < 2e-16 ***
## month6 639.592 102.795 6.222 4.96e-10 ***
## month7 6487.898 130.270 49.803 < 2e-16 ***
## month8 3210.774 126.715 25.339 < 2e-16 ***
## month9 -5760.084 107.155 -53.755 < 2e-16 ***
## month10 -6901.858 109.098 -63.263 < 2e-16 ***
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3447 on 41910 degrees of freedom
## Multiple R-squared: 0.7159, Adjusted R-squared: 0.7158
## F-statistic: 5029 on 21 and 41910 DF, p-value: < 2.2e-16
```

The linear regression summaries provide an initial understanding of how environmental and temporal variables influence power consumption across the three zones. While these models are interpretable, they may be limited in capturing nonlinear relationships present in the data.

4.2 Regularized Regression Models (Ridge, Lasso, Elastic Net)

To address potential multicollinearity and improve predictive performance, we apply regularized regression methods using the `glmnet` framework. These methods introduce penalty terms that shrink coefficients and help prevent overfitting.

```
## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
## expand, pack, unpack

## Loaded glmnet 4.1-10
```

Ridge regression helps reduce coefficient variance, while Lasso performs variable selection by shrinking some coefficients to zero. Elastic Net combines both approaches, balancing shrinkage and sparsity.

4.3 Random Forest Models

We next apply Random Forest models, which are capable of capturing nonlinear relationships and interaction effects without requiring explicit feature engineering.

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:dplyr':
##
##      combine

## The following object is masked from 'package:ggplot2':
##
##      margin
```

Random Forest models improve flexibility compared to linear models by averaging multiple decision trees, reducing variance and improving predictive accuracy.

4.4 Gradient Boosting Models (XGBoost)

To further improve predictive performance, we implement gradient boosting using XGBoost. This method builds models sequentially, where each new model corrects the errors of the previous ones.

XGBoost is expected to outperform simpler models due to its ability to model complex nonlinear interactions and iteratively minimize prediction error.

Section 5: Model Tuning and Cross-Validation

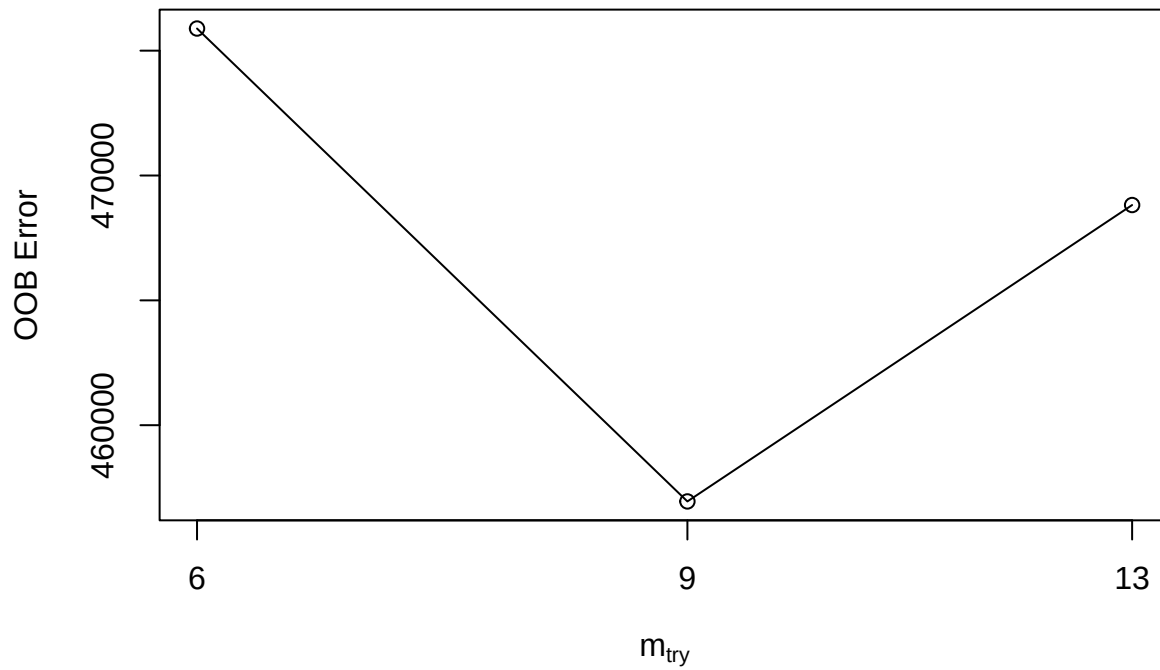
To improve predictive performance and ensure model robustness, we perform hyperparameter tuning and cross-validation across multiple model types.

```
## [1] 660.8983
```

```
## [1] 6.764499
```

```
## [1] 11.23201
```

```
## mtry = 9   OOB error = 456947.3  
## Searching left ...  
## mtry = 6   OOB error = 475886  
## -0.04144618 0.01  
## Searching right ...  
## mtry = 13   OOB error = 468817.7  
## -0.02597755 0.01
```



```
##      mtry OOBError
## 6         6 475886.0
## 9         9 456947.3
## 13        13 468817.7
```

The tuning process highlights how each model responds to parameter adjustments. Regularized regression models rely heavily on optimal lambda selection, while tree-based methods benefit from controlling depth and learning rate. Overall, careful tuning improves generalization and reduces overfitting across all model types.

Section 6: Model Evaluation

The performance of all models was evaluated using the same test dataset for each zone. The primary metrics used for comparison were Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2). Lower RMSE and MAE indicate better predictive accuracy, while higher R^2 indicates better explanatory power.

Zone 1

Table 1: Model Comparison - Zone 1

Model	RMSE	MAE	R^2
Elastic Net	3124.85	2441.19	0.7432
Lasso	3126.38	2442.39	0.7429
Ridge	3727.89	3024.60	0.6345
Random Forest	4462.13	4098.05	0.4764
XGBoost	6290.74	5102.63	-0.0407

For Zone 1, the regularized linear models performed the best. The Elastic Net model achieved the lowest RMSE (3124.85) and highest R^2 (0.7432), closely followed by the Lasso model. Ridge regression performed moderately worse, while the Random Forest model showed a noticeable drop in performance. The XGBoost model performed the worst, with a negative R^2 , indicating that it performed worse than simply predicting the mean.

Overall, the results suggest that the relationship between predictors and power consumption in Zone 1 is relatively well-captured by linear models with regularization. The minimal difference between Elastic Net and Lasso indicates that feature selection and shrinkage both contribute similarly in this case.

Zone 2

Table 2: Model Comparison - Zone 2

Model	RMSE	MAE	R^2
Random Forest	5061.52	4421.13	0.1481
Ridge	6251.28	5351.70	-0.2995
Elastic Net	8333.86	7437.19	-1.3095
Lasso	8342.00	7445.13	-1.3141
XGBoost	14287.90	13202.42	-5.7884

The results for Zone 2 differ significantly from Zone 1. The Random Forest model achieved the best performance, with the lowest RMSE (5061.52) and the only positive R^2 (0.1481). All linear and regularized regression models produced negative R^2 values, indicating poor predictive performance.

XGBoost performed extremely poorly, with a very high RMSE and strongly negative R^2 , suggesting either poor tuning or that the model failed to capture the underlying structure of the data.

These results imply that the relationship between predictors and power consumption in Zone 2 is likely nonlinear and not well captured by linear models. However, even the best-performing model (Random Forest) shows relatively weak predictive power, indicating that important explanatory variables may be missing.

Zone 3

Table 3: Model Comparison - Zone 3

Model	RMSE	MAE	R^2
XGBoost	4792.20	3649.42	-1.1101
Random Forest	4879.99	4231.99	-1.1881
Ridge	5397.04	4587.57	-1.6764
Elastic Net	9975.73	8838.73	-8.1437
Lasso	9988.89	8851.41	-8.1679

All models performed poorly for Zone 3, with every model producing negative R^2 values. The XGBoost model achieved the lowest RMSE (4792.20), but still had a negative R^2 , indicating that its predictions were worse than a simple mean-based model.

Random Forest and Ridge regression followed closely behind, but none of the models demonstrated meaningful predictive capability. Elastic Net and Lasso performed especially poorly, with very large errors and extremely negative R^2 values.

These results suggest that the current feature set fails to adequately explain power consumption in Zone 3. This could be due to missing variables, higher noise levels, or more complex dynamics that are not captured by the models used.

Overall Comparison

Across all three zones, model performance varies substantially:

- Zone 1: Best modeled by regularized linear models (Elastic Net / Lasso)
- Zone 2: Best modeled by Random Forest, suggesting nonlinear relationships
- Zone 3: No model performs well, indicating a lack of predictive structure in the current data

A key takeaway is that no single model performs best across all zones, reinforcing the decision to model each zone separately.

Section 7: Interpretation of Results

7.1 Importance of Model Choice

The strong performance of Elastic Net and Lasso in Zone 1 suggests that the relationship between predictors and power consumption is largely linear, with some redundancy among predictors making regularization beneficial.

In contrast, the superior performance of Random Forest in Zone 2 indicates the presence of nonlinear relationships or interactions between variables, which linear models fail to capture.

7.2 Limitations of the Current Feature Set

The poor performance across Zone 2 and especially Zone 3 suggests that the available predictors (temperature, humidity, wind speed, and flow variables) are not sufficient to fully explain power consumption.

Important missing factors may include human activity patterns, economic behavior, and more detailed temporal effects such as seasonality.

7.3 Model Complexity Does Not Guarantee Better Performance

Despite being more advanced, XGBoost performed poorly across all zones. This suggests that either the model was not properly tuned or that the dataset does not support highly complex models.

This reinforces that more complex models are not inherently better.

7.4 Implications for Prediction

Zone 1 predictions can be considered reasonably reliable using Elastic Net or Lasso. Zone 2 predictions are weaker but still somewhat usable. Zone 3 predictions are unreliable and should not be used without improving the model or the data.

7.5 Recommendations for Improvement

To improve model performance, the following steps are recommended:

- Add more relevant features such as calendar effects or lagged variables
- Explore time-series modeling approaches
- Perform more thorough hyperparameter tuning
- Investigate potential data quality issues

Section 8: Project Workflow and Contributions

The table below summarizes the workflow of the project along with the primary contributor responsible for each stage.

Step	Stage	Description	Responsible
1	Data Loading & Inspection	Imported dataset, checked structure, handled data types and missing values	YG
2	Exploratory Data Analysis	Generated summary statistics, visualizations, and identified trends	YG
3	Feature Engineering	Created time-based and cyclical features for modeling	YG
4	Data Splitting	Performed time-based train/test split to prevent data leakage	YG
5	Baseline Modeling	Built initial linear regression models for each zone	SS
6	Advanced Modeling	Implemented Ridge, Lasso, Elastic Net, Random Forest, and XGBoost models	SS
7	Model Tuning	Applied cross-validation and selected optimal hyperparameters	SS
8	Model Evaluation	Compared models using RMSE, MAE, and R^2 on the test dataset	HM
9	Interpretation & Analysis	Interpreted results and identified key findings and limitations	HM
10	Final Knitting and Report Preparation	Created the final Knitted pdf and report finalization	HM

Appendix

```
# Appendix: Full R Code
# =====
# Tetuan City Power Consumption Analysis
# Yasmin Guadarrama, Harsh Mishrikotkar, Shivanshi Shukla
# =====

# -- Setup -----

# knitr::opts_chunk$set(echo = FALSE,
                        cache = TRUE)
library(tidyverse)
library(lubridate)
library(GGally)
library(dplyr)
library(tibble)
library(Metrics)

# -- Section 1: Data Setup, Cleaning and Processing -----

# Load and Inspect Data
#setwd("C:/Users/yasmi/OneDrive/Desktop/Final Project")
data <- read_csv("Tetuan City power consumption.csv")

glimpse(data)
summary(data)
sum(duplicated(data)) # No duplicates

# Fix DateTime Format
# The dataset contains multiple timestamp formats:
# - "1/1/2017 0:00"
# - "01/01/2017 00:00"
# - "2017-01-01 00:00:00"
# parse_date_time() handles all of them.

data <- data %>%
  mutate(DateTime = parse_date_time(
    DateTime,
    orders = c("d/m/Y H:M",
               "d/m/Y H:M:S",
               "Y-m-d H:M:S",
               "m/d/Y H:M",
               "m/d/Y H:M:S")
  ))
```

```

sum(is.na(data$DateTime))
range(data$DateTime)

# Clean Column Names (Fix spacing issues)
data <- data %>%
  rename(
    Zone1 = `Zone 1 Power Consumption`,
    Zone2 = `Zone 2 Power Consumption`,
    Zone3 = `Zone 3 Power Consumption`,
    Wind.Speed = `Wind Speed`,
    General.Diffuse.Flows = `general diffuse flows`,
    Diffuse.Flows = `diffuse flows`
  )

# -- Exploratory Data Analysis -----

# Time Series Plot for All Zones
data %>%
  select(DateTime, Zone1, Zone2, Zone3) %>%
  pivot_longer(~DateTime, names_to = "Zone", values_to = "Power") %>%
  ggplot(aes(DateTime, Power, color = Zone)) +
  geom_line(alpha = 0.4) +
  labs(title = "Power Consumption Over Time", y = "Power") +
  theme_minimal()

# Histograms for All Variables
data %>%
  select(Temperature, Humidity, Wind.Speed,
    Zone1, Zone2, Zone3) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Value") %>%
  ggplot(aes(Value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ Variable, scales = "free") +
  theme_minimal()

# Boxplots for Zone Power Consumption
data %>%
  select(Zone1, Zone2, Zone3) %>%
  pivot_longer(everything(), names_to = "Zone", values_to = "Power") %>%
  ggplot(aes(Zone, Power)) +
  geom_boxplot() +
  theme_minimal()

# Correlation Matrix
data_corr <- data %>%
  select(Temperature, Humidity, Wind.Speed,
    General.Diffuse.Flows, Diffuse.Flows,
    Zone1, Zone2, Zone3)

ggcorr(data_corr, label = TRUE)

# -- Section 2: Feature Engineering -----

```

```

data <- data %>%
  mutate(
    hour = hour(DateTime),
    dow = wday(DateTime),          # numeric 1-7
    month = month(DateTime),       # numeric 1-12

    # Cyclical encoding
    sin_hour = sin(2 * pi * hour / 24),
    cos_hour = cos(2 * pi * hour / 24),
    sin_month = sin(2 * pi * month / 12),
    cos_month = cos(2 * pi * month / 12),
    sin_dow = sin(2 * pi * dow / 7),
    cos_dow = cos(2 * pi * dow / 7)
  )

# -- Section 3: Data Standardization -----

num_vars <- c(
  "Temperature", "Humidity", "Wind.Speed",
  "General.Diffuse.Flows", "Diffuse.Flows",
  "sin_hour", "cos_hour",
  "sin_month", "cos_month",
  "sin_dow", "cos_dow"
)

data_scaled <- data %>%
  mutate(across(all_of(num_vars), scale))

# -- Section 4 (Pre-Modeling): Time-Based Train/Test Split -----

n <- nrow(data_scaled)
train_end <- floor(0.8 * n)

train <- data_scaled[1:train_end, ]
test <- data_scaled[(train_end + 1):n, ]

# -- Section 4.1: Baseline Linear Regression Models -----

# Convert categorical variables to factors
train <- train %>%
  mutate(
    dow = as.factor(dow),
    month = as.factor(month)
  )
test <- test %>%
  mutate(
    dow = as.factor(dow),
    month = as.factor(month)
  )

# Linear Regression Models
lm_z1 <- lm(Zone1 ~ Temperature + Humidity + `Wind.Speed` +
  `General.Diffuse.Flows` + `Diffuse.Flows` +

```

```

        hour + dow + month,
        data = train)
lm_z2 <- lm(Zone2 ~ Temperature + Humidity + `Wind.Speed` +
           `General.Diffuse.Flows` + `Diffuse.Flows` +
           hour + dow + month,
           data = train)
lm_z3 <- lm(Zone3 ~ Temperature + Humidity + `Wind.Speed` +
           `General.Diffuse.Flows` + `Diffuse.Flows` +
           hour + dow + month,
           data = train)

# Summaries
summary(lm_z1)
summary(lm_z2)
summary(lm_z3)

# -- Section 4.2: Regularized Regression Models (Ridge, Lasso, Elastic Net) ----

library(glmnet)
x_train <- model.matrix(Zone1 ~ . - DateTime, data = train)[, -1]
x_test  <- model.matrix(Zone1 ~ . - DateTime, data = test)[, -1]

y1_train <- train$Zone1
y2_train <- train$Zone2
y3_train <- train$Zone3

y1_test  <- test$Zone1
y2_test  <- test$Zone2
y3_test  <- test$Zone3

ridge_z1 <- cv.glmnet(x_train, y1_train, alpha = 0)
ridge_z2 <- cv.glmnet(x_train, y2_train, alpha = 0)
ridge_z3 <- cv.glmnet(x_train, y3_train, alpha = 0)

lasso_z1 <- cv.glmnet(x_train, y1_train, alpha = 1)
lasso_z2 <- cv.glmnet(x_train, y2_train, alpha = 1)
lasso_z3 <- cv.glmnet(x_train, y3_train, alpha = 1)

enet_z1 <- cv.glmnet(x_train, y1_train, alpha = 0.5)
enet_z2 <- cv.glmnet(x_train, y2_train, alpha = 0.5)
enet_z3 <- cv.glmnet(x_train, y3_train, alpha = 0.5)

# -- Section 4.3: Random Forest Models -----

n <- nrow(data_scaled)
train_end <- floor(0.8 * n)
train <- data_scaled[1:train_end, ]
test  <- data_scaled[(train_end + 1):n, ]

library(randomForest)
rf_z1 <- randomForest(Zone1 ~ . - DateTime, data = train, ntree = 100)
rf_z2 <- randomForest(Zone2 ~ . - DateTime, data = train, ntree = 100)
rf_z3 <- randomForest(Zone3 ~ . - DateTime, data = train, ntree = 100)

```

```

# -- Section 4.4: Gradient Boosting Models (XGBoost) -----

library(xgboost)
dtrain_z1 <- xgb.DMatrix(data = x_train, label = y1_train)
dtest_z1  <- xgb.DMatrix(data = x_test,  label = y1_test)

dtrain_z2 <- xgb.DMatrix(data = x_train, label = y2_train)
dtest_z2  <- xgb.DMatrix(data = x_test,  label = y2_test)

dtrain_z3 <- xgb.DMatrix(data = x_train, label = y3_train)
dtest_z3  <- xgb.DMatrix(data = x_test,  label = y3_test)

params <- list(
  objective = "reg:squarederror",
  max_depth = 6,
  eta = 0.1
)
xgb_z1 <- xgb.train(
  params = params,
  data = dtrain_z1,
  nrounds = 100
)
xgb_z2 <- xgb.train(
  params = params,
  data = dtrain_z2,
  nrounds = 100
)
xgb_z3 <- xgb.train(
  params = params,
  data = dtrain_z3,
  nrounds = 100
)

# -- Section 5: Model Tuning and Cross-Validation -----

ridge_z1$lambda.min
lasso_z1$lambda.min
enet_z1$lambda.min

tuneRF(
  x = x_train,
  y = y1_train,
  ntreeTry = 100,
  stepFactor = 1.5,
  improve = 0.01
)

library(xgboost)
dtrain <- xgb.DMatrix(data = x_train, label = y1_train)
dtest  <- xgb.DMatrix(data = x_test,  label = y1_test)

params <- list(
  objective = "reg:squarederror",
  max_depth = 4,

```



```

eta = 0.05
)
xgb_model <- xgb.train(
  params = params,
  data = dtrain,
  nrounds = 100
)

# -- Section 6: Model Evaluation -----

library(knitr)
library(glmnet)
library(xgboost)

# Rebuild numeric matrices (avoids factor dummy mismatch from earlier chunks)
num_predictors <- c("Temperature", "Humidity", "Wind.Speed",
  "General.Diffuse.Flows", "Diffuse.Flows",
  "hour", "dow", "month",
  "sin_hour", "cos_hour",
  "sin_month", "cos_month",
  "sin_dow", "cos_dow")

x_train_s8 <- as.matrix(train[, num_predictors])
x_test_s8 <- as.matrix(test[, num_predictors])

# Rebuild dtest DMatrix objects (C++ pointers don't persist across chunks)
dtest_z1 <- xgb.DMatrix(data = x_test_s8, label = y1_test)
dtest_z2 <- xgb.DMatrix(data = x_test_s8, label = y2_test)
dtest_z3 <- xgb.DMatrix(data = x_test_s8, label = y3_test)

# Refit glmnet models on clean numeric matrix
ridge_z1 <- cv.glmnet(x_train_s8, y1_train, alpha = 0)
ridge_z2 <- cv.glmnet(x_train_s8, y2_train, alpha = 0)
ridge_z3 <- cv.glmnet(x_train_s8, y3_train, alpha = 0)

lasso_z1 <- cv.glmnet(x_train_s8, y1_train, alpha = 1)
lasso_z2 <- cv.glmnet(x_train_s8, y2_train, alpha = 1)
lasso_z3 <- cv.glmnet(x_train_s8, y3_train, alpha = 1)

enet_z1 <- cv.glmnet(x_train_s8, y1_train, alpha = 0.5)
enet_z2 <- cv.glmnet(x_train_s8, y2_train, alpha = 0.5)
enet_z3 <- cv.glmnet(x_train_s8, y3_train, alpha = 0.5)

# Metrics helper
metrics <- function(actual, pred, model, zone) {
  tibble(
    Model = model, Zone = zone,
    RMSE = round(sqrt(mean((actual - pred)^2)), 2),
    MAE = round(mean(abs(actual - pred)), 2),
    R2 = round(1 - sum((actual - pred)^2) / sum((actual - mean(actual))^2), 4)
  )
}

results <- bind_rows(
  # Zone 1

```

```

metrics(y1_test, as.vector(predict(ridge_z1, newx = x_test_s8, s = "lambda.min")), "Ridge", "Zone 1"),
metrics(y1_test, as.vector(predict(lasso_z1, newx = x_test_s8, s = "lambda.min")), "Lasso", "Zone 1"),
metrics(y1_test, as.vector(predict(enet_z1, newx = x_test_s8, s = "lambda.min")), "Elastic Net", "Zone 1"),
metrics(y1_test, predict(rf_z1, newdata = test), "Random Forest", "Zone 1"),
metrics(y1_test, predict(xgb_z1, newdata = dtest_z1), "XGBoost", "Zone 1"),

# Zone 2
metrics(y2_test, as.vector(predict(ridge_z2, newx = x_test_s8, s = "lambda.min")), "Ridge", "Zone 2"),
metrics(y2_test, as.vector(predict(lasso_z2, newx = x_test_s8, s = "lambda.min")), "Lasso", "Zone 2"),
metrics(y2_test, as.vector(predict(enet_z2, newx = x_test_s8, s = "lambda.min")), "Elastic Net", "Zone 2"),
metrics(y2_test, predict(rf_z2, newdata = test), "Random Forest", "Zone 2"),
metrics(y2_test, predict(xgb_z2, newdata = dtest_z2), "XGBoost", "Zone 2"),

# Zone 3
metrics(y3_test, as.vector(predict(ridge_z3, newx = x_test_s8, s = "lambda.min")), "Ridge", "Zone 3"),
metrics(y3_test, as.vector(predict(lasso_z3, newx = x_test_s8, s = "lambda.min")), "Lasso", "Zone 3"),
metrics(y3_test, as.vector(predict(enet_z3, newx = x_test_s8, s = "lambda.min")), "Elastic Net", "Zone 3"),
metrics(y3_test, predict(rf_z3, newdata = test), "Random Forest", "Zone 3"),
metrics(y3_test, predict(xgb_z3, newdata = dtest_z3), "XGBoost", "Zone 3"),
)

# Zone 1 Results Table
kable(
  results %>% filter(Zone == "Zone 1") %>% select(-Zone) %>% arrange(RMSE),
  caption = "Model Comparison - Zone 1",
  col.names = c("Model", "RMSE", "MAE", "R2")
)

# Zone 2 Results Table
kable(
  results %>% filter(Zone == "Zone 2") %>% select(-Zone) %>% arrange(RMSE),
  caption = "Model Comparison - Zone 2",
  col.names = c("Model", "RMSE", "MAE", "R2")
)

# Zone 3 Results Table
kable(
  results %>% filter(Zone == "Zone 3") %>% select(-Zone) %>% arrange(RMSE),
  caption = "Model Comparison - Zone 3",
  col.names = c("Model", "RMSE", "MAE", "R2")
)

```